

▼ Aula 4 — Construindo um mini-RAG passo a passo

Nesta atividade, a gente vai construir um pipeline simples de **RAG**.
RAG significa **Retrieval-Augmented Generation**, ou seja:

 Geração aumentada por recuperação de informação.

A ideia é simples:

- 1. Temos documentos.
- 2. Quebramos esses documentos em pedaços menores.
- 3. Transformamos esses pedaços em vetores.
- 4. Buscamos os pedaços mais relevantes para uma pergunta.
- 5. Usamos esses pedaços como contexto para montar uma resposta.

Nesta prática, a gente não vai depender de API paga nem de um LLM externo.
O foco aqui é entender o funcionamento do pipeline.
Depois, em um projeto maior, você pode trocar algumas partes por LangChain, LlamaIndex, OpenAI, Hugging Face, Chroma, FAISS e outros componentes.

Objetivos da atividade

Ao final deste notebook, você deve conseguir:

- Explicar por que LLMs podem alucinar.
- Explicar o papel do RAG.
- Quebrar documentos em chunks.
- Criar uma representação vetorial simples dos textos.
- Buscar os chunks mais relevantes para uma pergunta.
- Montar um prompt com contexto recuperado.
- Avaliar se a recuperação está trazendo bons resultados.

A ideia não é decorar código.
A ideia é entender o fluxo.

▼ Antes de começar

Neste notebook, a gente vai usar uma abordagem didática.
Em vez de começar direto com LangChain ou LlamaIndex, vamos montar o pipeline quase “na mão”.
Isso ajuda a entender o que essas bibliotecas fazem por baixo dos panos.

A gente vai usar:

- `pandas`, para visualizar tabelas;
- `scikit-learn`, para criar uma representação vetorial simples;
- `cosine_similarity`, para comparar pergunta e documentos.

Aqui, o nosso “embedding” será baseado em TF-IDF.
Em sistemas reais, normalmente usamos embeddings neurais, como modelos da OpenAI, Sentence Transformers, BGE, E5, entre outros.
Mas para aprender o conceito, TF-IDF já ajuda bastante.

```
# Caso esteja rodando no Google Colab ou em um ambiente novo,
# descomente a linha abaixo.

# !pip install -q pandas numpy scikit-learn
```

```
import re
import textwrap
import numpy as np
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

Parte 1 — O problema

Imagine que um aluno pergunta para uma LLM:

 "O que foi ensinado na Aula 4?"

Se o modelo não tiver acesso ao nosso plano de aula, ele pode tentar responder com base no conhecimento geral dele.
E aí mora o perigo.
Ele pode responder algo plausível.
Bonito.
Bem escrito.
Com cara de certo.
Mas não necessariamente baseado no nosso material.
Esse é um dos problemas que o RAG tenta reduzir.

▼ Mini-base de conhecimento

Para esta atividade, vamos criar uma base pequena de documentos.

Ela vai representar, de forma simplificada, o conteúdo da Aula 4.

Em um cenário real, essa base poderia ser:

- um PDF;
- um conjunto de slides;
- uma documentação interna;
- artigos;
- contratos;
- prontuários;
- páginas de uma wiki;
- relatórios técnicos.

Aqui, para facilitar, vamos começar com textos pequenos.

```
documentos = [  
    {  
        "id": "doc_1",  
        "titulo": "Limitações dos LLMs",  
        "texto": """"  
        Modelos de linguagem podem gerar respostas fluentes mesmo quando não sabem a resposta correta.  
        Esse fenômeno é conhecido como alucinação. Além disso, LLMs podem ter conhecimento congelado,  
        pois foram treinados até uma determinada data. Outra limitação importante é a janela de contexto,  
        que define quanto texto o modelo consegue considerar de uma vez.  
        """"  
    },  
    {  
        "id": "doc_2",  
        "titulo": "O que é RAG",  
        "texto": """"  
        RAG significa Retrieval-Augmented Generation. A ideia é combinar recuperação de informação  
        com geração de texto. Antes de responder, o sistema busca trechos relevantes em uma base de conhecimento.  
        Depois, esses trechos são enviados como contexto para o modelo gerar uma resposta mais fundamentada.  
        """"  
    },  
    {  
        "id": "doc_3",  
        "titulo": "Embeddings e bancos vetoriais",  
        "texto": """"  
        Embeddings são representações numéricas de textos. Textos semanticamente parecidos tendem a ter  
        vetores próximos. Bancos vetoriais armazenam esses vetores e permitem buscar rapidamente os trechos  
        mais semelhantes a uma pergunta. Exemplos de bancos vetoriais incluem FAISS, Chroma, Pinecone,  
        Weaviate e Milvus.  
        """"  
    },  
    {  
        "id": "doc_4",  
        "titulo": "Pipeline RAG",  
        "texto": """"  
        Um pipeline RAG geralmente possui etapas de indexação, chunking, criação de embeddings,  
        armazenamento em banco vetorial, recuperação dos trechos mais relevantes e geração da resposta.  
        Durante a indexação, os documentos são preparados. Durante a recuperação, a pergunta do usuário  
        também é transformada em vetor para buscar os chunks mais próximos.  
        """"  
    },  
    {  
        "id": "doc_5",  
        "titulo": "Avaliação de RAG",  
        "texto": """"  
        Avaliar um sistema RAG envolve verificar se os trechos recuperados são relevantes,  
        se a resposta usa o contexto recuperado e se a resposta final está correta.  
        Algumas dimensões importantes são relevância da recuperação, fidelidade ao contexto,  
        cobertura da resposta e ausência de alucinação.  
        """"  
    }  
]
```

```
df_documentos = pd.DataFrame(documentos)  
df_documentos
```

Exercício rápido 1

Antes de escrever qualquer código de RAG, responda:

1. Por que um LLM pode errar ao responder sobre um documento específico?
2. O que muda quando damos ao modelo um contexto recuperado de uma base?
3. RAG elimina completamente o risco de alucinação?

Escreva sua resposta na célula abaixo.

Sua resposta

Escreva aqui:

- 1.
- 2.
- 3.

Parte 2 — Chunking

Agora vamos simular uma etapa muito importante do RAG:

quebrar documentos grandes em pedaços menores.

Esses pedaços são chamados de **chunks**.

Por que fazer isso?

Porque normalmente não queremos enviar um documento inteiro para o modelo.

A gente quer recuperar apenas os pedaços mais relevantes.

Um chunk muito grande pode trazer informação demais.

Um chunk muito pequeno pode perder contexto.

Esse equilíbrio é uma das partes mais importantes de um bom sistema RAG.

```
def limpar_texto(texto):
    """
    Remove espaços extras e quebras de linha desnecessárias.
    """
    texto = texto.strip()
    texto = re.sub(r"\s+", " ", texto)
    return texto

def criar_chunks_por_palavras(texto, tamanho_chunk=40, sobreposicao=10):
    """
    Divide um texto em chunks com base em quantidade de palavras.

    Parâmetros:
    - texto: texto original.
    - tamanho_chunk: número máximo de palavras por chunk.
    - sobreposicao: quantidade de palavras repetidas entre chunks.

    Retorna:
    - lista de chunks.
    """
    texto = limpar_texto(texto)
    palavras = texto.split()

    chunks = []
    inicio = 0

    while inicio < len(palavras):
        fim = inicio + tamanho_chunk
        chunk = " ".join(palavras[inicio:fim])
        chunks.append(chunk)

        if fim >= len(palavras):
            break

        inicio = fim - sobreposicao

    return chunks
```

```
# Testando o chunking em um documento

texto_teste = documentos[3]["texto"]

chunks_teste = criar_chunks_por_palavras(
    texto_teste,
    tamanho_chunk=25,
    sobreposicao=5
)

for i, chunk in enumerate(chunks_teste):
    print(f"Chunk {i+1}")
    print(chunk)
    print("-" * 80)
```

Exercício 2 — Testando tamanhos de chunk

Altere os valores de tamanho_chunk e sobreposicao na célula abaixo.

Teste pelo menos três combinações:

- chunks pequenos;
- chunks médios;
- chunks maiores.

Depois, responda:

Qual configuração parece preservar melhor o sentido do texto?

```
#! TODO: teste diferentes valores aqui

tamanho_chunk = 30
sobreposicao = 5

chunks_experimento = criar_chunks_por_palavras(
    documentos[3]["texto"],
    tamanho_chunk=tamanho_chunk,
    sobreposicao=sobreposicao
)

for i, chunk in enumerate(chunks_experimento):
    print(f"Chunk {i+1}: {chunk}")
    print()
```

Sua conclusão

Escreva aqui:

- O que aconteceu quando o chunk ficou pequeno demais?
- O que aconteceu quando o chunk ficou grande demais?
- Para esse exemplo, qual configuração você escolheria?

Parte 3 — Criando os chunks da nossa base

Agora vamos aplicar o chunking em todos os documentos.

Cada chunk vai guardar:

- o texto do chunk;
- o documento de origem;
- o título do documento;
- o índice do chunk dentro daquele documento.

Isso é importante porque, em sistemas RAG reais, a gente quase sempre precisa saber de onde veio cada trecho recuperado.

```
chunks = []

for doc in documentos:
    lista_chunks = criar_chunks_por_palavras(
        doc["texto"],
        tamanho_chunk=35,
        sobreposicao=8
    )

    for i, chunk in enumerate(lista_chunks):
        chunks.append({
            "chunk_id": f"{doc['id']}_chunk_{i+1}",
            "doc_id": doc["id"],
            "titulo": doc["titulo"],
            "chunk_index": i + 1,
            "texto": chunk
        })

df_chunks = pd.DataFrame(chunks)
df_chunks
```

```
print(f"Quantidade de documentos: {len(documentos)}")
print(f"Quantidade de chunks: {len(chunks)}")
```

▼ Parte 4 — Transformando texto em vetor

Agora entra a parte dos embeddings.

Em um RAG real, usáramos um modelo de embeddings para transformar cada chunk em um vetor.

Aqui, para fins didáticos, vamos usar TF-IDF.

O TF-IDF cria uma representação numérica baseada nas palavras do texto.

Não é tão poderoso quanto embeddings neurais, mas já permite fazer uma busca inicial por similaridade.

```
textos_chunks = df_chunks["texto"].tolist()

vectorizer = TfidfVectorizer()
matriz_chunks = vectorizer.fit_transform(textos_chunks)

matriz_chunks.shape
```

O resultado acima mostra o formato da matriz.

A primeira dimensão representa a quantidade de chunks.

A segunda dimensão representa a quantidade de termos do vocabulário aprendido pelo `TfidfVectorizer`.

Cada chunk virou um vetor.

Agora a gente consegue comparar uma pergunta com esses chunks.

```
vocabulario = vectorizer.get_feature_names_out()

print("Quantidade de termos no vocabulário:", len(vocabulario))
print("Primeiros termos:")
print(vocabulario[:30])
```

▼ Parte 5 — Recuperação

Agora vamos fazer a etapa de retrieval.

A lógica é:

1. Receber uma pergunta.
2. Transformar a pergunta em vetor.
3. Comparar esse vetor com os vetores dos chunks.
4. Retornar os chunks mais parecidos.

Para medir similaridade, vamos usar similaridade do cosseno.

```
def recuperar_chunks(pergunta, top_k=3):
    """
    Recupera os chunks mais relevantes para uma pergunta.
    """
    vetor_pergunta = vectorizer.transform([pergunta])

    similaridades = cosine_similarity(vetor_pergunta, matriz_chunks)[0]

    indices_ordenados = np.argsort(similaridades)[::-1]

    resultados = []

    for idx in indices_ordenados[:top_k]:
        resultados.append({
            "chunk_id": df_chunks.iloc[idx]["chunk_id"],
            "titulo": df_chunks.iloc[idx]["titulo"],
            "texto": df_chunks.iloc[idx]["texto"],
            "score": similaridades[idx]
        })

    return pd.DataFrame(resultados)
```

```
pergunta = "O que é RAG e como ele ajuda a reduzir alucinações?"
```

```
resultados = recuperar_chunks(pergunta, top_k=3)
resultados
```

Exercício 3 — Investigando a recuperação

Teste pelo menos 4 perguntas diferentes.

Sugestões:

- O que é janela de contexto?
- Para que servem embeddings?
- Quais são as etapas de um pipeline RAG?
- Como avaliar um sistema RAG?

Depois, observe:

- O chunk recuperado faz sentido?
- O score foi alto ou baixo?
- Algum chunk irrelevante apareceu?

```
#! TODO: escreva sua pergunta aqui

minha_pergunta = "?"

recuperar_chunks(minha_pergunta, top_k=3)
```

Sua análise

Escreva aqui:

- A recuperação funcionou bem?
- O primeiro resultado era realmente o melhor?
- O que poderia melhorar?

Parte 6 — Montando um prompt com contexto

Até agora, fizemos a parte de recuperação.

Mas RAG não é só buscar texto.

A ideia é buscar texto e usar esse texto para ajudar um modelo a responder.

O prompt geralmente tem esta estrutura:

- Instrução para o modelo.
- Contexto recuperado.
- Pergunta do usuário.
- Regras de resposta.

Exemplo:

Responda usando apenas o contexto abaixo.
Se o contexto não for suficiente, diga que não há informação suficiente.

```
def montar_contexto(df_resultados):
    """
    Monta um bloco de contexto a partir dos chunks recuperados.
    """
    partes = []

    for i, row in df_resultados.iterrows():
        parte = f"""
Fonte: {row['titulo']} | Chunk: {row['chunk_id']}
Texto: {row['texto']}
"""
        partes.append(parte.strip())

    return "\n\n".join(partes)

def montar_prompt(pergunta, df_resultados):
    """
    Monta um prompt no estilo RAG.
    """
    contexto = montar_contexto(df_resultados)

    prompt = f"""
Você é um assistente da disciplina de RNN e Transformadores.

Responda à pergunta usando apenas o contexto fornecido.

Se o contexto não tiver informação suficiente, diga:
"Não encontrei informação suficiente no contexto."

Contexto:
{contexto}

Pergunta:
{pergunta}

Resposta:
"""
    return prompt.strip()
```

```
pergunta = "Quais são as etapas de um pipeline RAG?"

resultados = recuperar_chunks(pergunta, top_k=3)
prompt = montar_prompt(pergunta, resultados)

print(prompt)
```

Pausa rápida

Perceba uma coisa importante:

A gente ainda não chamou nenhum LLM.

Mas já construímos uma parte enorme do RAG:

- documentos;
- chunks;
- vetores;
- busca por similaridade;
- contexto;
- prompt.

Em muitos projetos, o problema não está só no modelo.

Está na qualidade da recuperação.

Um RAG com recuperação ruim manda contexto ruim para o LLM.

E contexto ruim gera resposta ruim.

É aquele famoso caso:

entra bagunça, sai bagunça.

Só que agora com um texto bonito.

Parte 7 — Simulando uma resposta

Como este notebook não depende de API externa, vamos criar uma resposta simples baseada nos chunks.

Ela não será uma resposta gerada por um LLM.

Será uma resposta didática, só para simular a etapa final.

A função vai:

1. recuperar os chunks;
2. montar o contexto;
3. devolver uma resposta simples com base nos trechos encontrados.

Em um projeto real, essa parte seria substituída por uma chamada a um LLM.

```
def responder_com_rag_simples(pergunta, top_k=3):
    """
    Simula uma resposta RAG sem chamar um LLM externo.
    """
    resultados = recuperar_chunks(pergunta, top_k=top_k)

    resposta = f"Pergunta: {pergunta}\n\n"
    resposta += "Com base nos trechos recuperados, encontrei estas informações:\n\n"

    for i, row in resultados.iterrows():
        resposta += f"- Fonte: {row['titulo']} ({row['chunk_id']})\n"
        resposta += f" Trecho: {row['texto']}\n\n"

    resposta += "Em um sistema real, esses trechos seriam enviados para um LLM gerar uma resposta final mais natural."

    return resposta
```

```
print(responder_com_rag_simples("Como o RAG ajuda a lidar com alucinações?", top_k=3))
```

Exercício 4 — Mudando o top_k

O parâmetro `top_k` controla quantos chunks serão recuperados.

Teste a mesma pergunta com:

- `top_k=1`
- `top_k=2`
- `top_k=4`

Depois responda:

- Com `top_k=1`, faltou informação?
- Com `top_k=4`, apareceu ruído?
- Qual valor parece melhor para esta base pequena?

```
pergunta = "Como avaliar um sistema RAG?"

# Coloque para o valor do TOP K ser 1,2,4
for k in [1]:
    print("=" * 100)
    print(f"TOP_K = {k}")
    print("=" * 100)
    print(responder_com_rag_simples(pergunta, top_k=k))
    print()
```

Sua conclusão sobre top_k

Escreva aqui:

- Melhor valor de `top_k`:
- Por quê?
- O que pode acontecer se `top_k` for alto demais?
- O que pode acontecer se `top_k` for baixo demais?

Parte 8 — Criando uma avaliação simples

Agora vamos avaliar a recuperação.

Vamos criar um pequeno conjunto de perguntas de teste.

Para cada pergunta, vamos indicar qual documento deveria aparecer entre os resultados.

Isso não é uma avaliação perfeita.

Mas já ajuda a medir se o retrieval está indo para o caminho certo.

```
perguntas_teste = [
    {
        "pergunta": "O que é alucinação em LLMs?",
        "doc_esperado": "Limitações dos LLMs"
    },
    {
        "pergunta": "O que significa RAG?",
        "doc_esperado": "O que é RAG"
    },
    {
        "pergunta": "Para que servem embeddings?",
        "doc_esperado": "Embeddings e bancos vetoriais"
    },
    {
        "pergunta": "Quais são as etapas do pipeline RAG?",
        "doc_esperado": "Pipeline RAG"
    },
    {
        "pergunta": "Como avaliar um sistema RAG?",
        "doc_esperado": "Avaliação de RAG"
    }
]

df_teste = pd.DataFrame(perguntas_teste)
df_teste
```

```
def avaliar_retrieval(perguntas_teste, top_k=3):
    """
    Avalia se o documento esperado aparece entre os top_k resultados.
    """
    linhas = []

    for item in perguntas_teste:
        pergunta = item["pergunta"]
        doc_esperado = item["doc_esperado"]

        resultados = recuperar_chunks(pergunta, top_k=top_k)

        titulos_recuperados = resultados["titulo"].tolist()

        acertou = doc_esperado in titulos_recuperados

        linhas.append({
            "pergunta": pergunta,
            "doc_esperado": doc_esperado,
            "titulos_recuperados": titulos_recuperados,
            "acertou": acertou
        })

    return pd.DataFrame(linhas)
```

```
df_avaliacao = avaliar_retrieval(perguntas_teste, top_k=3)
df_avaliacao
```

```
acuracia = df_avaliacao["acertou"].mean()

print(f"Acurácia de recuperação@3: {acuracia:.2%}")
```

Exercício 5 — Avaliando com top_k diferente

Agora teste:

- top_k=1
- top_k=2
- top_k=3

Compare os resultados.

Pergunta:

Aumentar o top_k sempre melhora o sistema?

```
for k in [1, 2, 3]:
    df_eval_k = avaliar_retrieval(perguntas_teste, top_k=k)
    acc_k = df_eval_k["acertou"].mean()

    print(f"top_k={k} | acurácia={acc_k:.2%}")
```

Sua resposta

Escreva aqui:

- Aumentar top_k melhorou?
- Teve algum risco?
- Em um sistema real, como você escolheria esse valor?

Parte 9 — Um mini vector store

Até agora, usamos variáveis soltas:

- `vectorizer`
- `matriz_chunks`
- `df_chunks`
- função de recuperação

Agora vamos organizar isso em uma classe simples.

A ideia é simular, de forma didática, o papel de um banco vetorial.

Um banco vetorial real faz muito mais coisa.

Mas, conceitualmente, ele precisa:

1. receber textos;
2. transformar em vetores;
3. armazenar os vetores;
4. buscar textos semelhantes a uma pergunta.

```
class MiniVectorStore:
    def __init__(self):
        self.vectorizer = TfidfVectorizer()
        self.matriz = None
        self.df = None

    def fit(self, df_chunks):
        """
        Indexa os chunks.
        """
        self.df = df_chunks.reset_index(drop=True).copy()
        textos = self.df["texto"].tolist()
        self.matriz = self.vectorizer.fit_transform(textos)

    def search(self, query, top_k=3):
        """
        Busca chunks similares à query.
        """
        if self.matriz is None:
            raise ValueError("A base ainda não foi indexada. Use o método fit primeiro.")

        vetor_query = self.vectorizer.transform([query])
        similaridades = cosine_similarity(vetor_query, self.matriz)[0]

        indices = np.argsort(similaridades[::-1][:top_k])

        resultados = self.df.iloc[indices].copy()
        resultados["score"] = similaridades[indices]

        return resultados[["chunk_id", "titulo", "texto", "score"]]
```

```
store = MiniVectorStore()
store.fit(df_chunks)

store.search("O que é banco vetorial?", top_k=3)
```

Exercício 6 — Usando o MiniVectorStore

Use o `store.search()` para responder às perguntas abaixo:

1. O que é RAG?
2. O que são embeddings?
3. Quais são as limitações dos LLMs?
4. Como funciona o pipeline RAG?
5. Como avaliar RAG?

Para cada pergunta, observe se o primeiro chunk recuperado foi bom.

```
# TODO: teste as perguntas aqui

perguntas = [
    "O que é RAG?",
    "O que são embeddings?",
    "Quais são as limitações dos LLMs?",
    "Como funciona o pipeline RAG?",
    "Como avaliar RAG?"
]

for pergunta in perguntas:
    print("=" * 100)
    print("Pergunta:", pergunta)
    print("=" * 100)

    display(store.search(pergunta, top_k=2))
```

Parte 10 — O que acontece quando a pergunta está fora da base?

Um ponto importante em RAG:

Nem sempre a base tem a resposta.

Quando isso acontece, o sistema deveria assumir que não encontrou informação suficiente.

Esse comportamento é importante para reduzir alucinação.

Vamos testar uma pergunta que não está na nossa base.

```
pergunta_fora_da_base = "Qual foi a arquitetura exata do modelo GPT-4?"

store.search(pergunta_fora_da_base, top_k=3)
```

Observe que o sistema sempre vai retornar alguma coisa.

Mesmo que a pergunta esteja fora da base.

Isso é perigoso.

Um sistema de RAG precisa ter alguma estratégia para dizer:

“Não encontrei informação suficiente.”

Uma estratégia simples é usar um limiar mínimo de similaridade.

Se o score for muito baixo, o sistema não responde.

```
def responder_com_limiar(pergunta, top_k=3, limiar=0.15):
    resultados = store.search(pergunta, top_k=top_k)

    melhor_score = resultados.iloc[0]["score"]

    if melhor_score < limiar:
        return "Não encontrei informação suficiente no contexto."

    return responder_com_rag_simples(pergunta, top_k=top_k)

print(responder_com_limiar("O que é RAG?", top_k=3, limiar=0.15))

print(responder_com_limiar("Qual foi a arquitetura exata do modelo GPT-4?", top_k=3, limiar=0.15))
```

Exercício 7 — Ajustando o limiar

Teste diferentes valores para o limiar:

- 0.05
- 0.10
- 0.15
- 0.25
- 0.40

Pergunta:

- O que acontece quando o limiar fica baixo demais?
- O que acontece quando o limiar fica alto demais?

```
perguntas_para_testar = [
    "O que é RAG?",
    "Como avaliar um sistema RAG?",
    "Qual foi a arquitetura exata do GPT-4?",
    "Quem ganhou a Copa do Mundo de 2018?"
]

limiares = [0.05, 0.10, 0.15, 0.25, 0.40]

for limiar in limiares:
    print("=" * 100)
    print(f"LIMIAR = {limiar}")
    print("=" * 100)

    for pergunta in perguntas_para_testar:
        resultados = store.search(pergunta, top_k=1)
        score = resultados.iloc[0]["score"]
        resposta = responder_com_limiar(pergunta, top_k=3, limiar=limiar)

        print(f"Pergunta: {pergunta}")
        print(f"Melhor score: {score:.3f}")
        print(f"Resposta: {resposta[:120]}...")
        print()
```

Sua conclusão

Escreva aqui:

- Melhor limiar encontrado:
- Por quê?
- Qual risco de um limiar muito baixo?
- Qual risco de um limiar muito alto?

Parte 11 — Simulando um documento maior

Agora vamos criar um texto maior, como se fosse um trecho de material didático.

A ideia é aproximar um pouco mais do cenário de PDF.

Em vez de vários documentos pequenos, teremos um documento maior que precisa ser quebrado em chunks.

```
material_aula_4 = """
A Aula 4 da disciplina de Redes Neurais Recorrentes e Transformadores tem como tema
Potencializando LLMs com Retrieval-Augmented Generation, também conhecido como RAG.

A aula começa discutindo as limitações dos grandes modelos de linguagem. Embora LLMs sejam
capazes de gerar textos fluentes e úteis, eles podem apresentar alucinações. Uma alucinação
ocorre quando o modelo produz uma resposta que parece correta, mas não está fundamentada em
informações verdadeiras ou disponíveis no contexto.

Outra limitação importante é o conhecimento congelado. Um modelo treinado até uma certa data
não conhece automaticamente eventos, documentos ou informações criadas depois do seu treinamento.
Além disso, mesmo quando a informação existe, ela pode não estar presente na janela de contexto.

A janela de contexto representa a quantidade máxima de texto que o modelo consegue considerar
durante a geração. Se um documento for muito grande, talvez não seja possível enviar tudo ao
modelo de uma vez. Por isso, precisamos recuperar apenas os trechos mais relevantes.

RAG é uma arquitetura que combina recuperação de informação com geração de texto. Em vez de
```

pedir que o modelo responda apenas com base no que aprendeu no treinamento, o sistema primeiro busca trechos relevantes em uma base de conhecimento. Depois, esses trechos são enviados como contexto para o modelo gerar a resposta.

Um pipeline RAG normalmente começa com a indexação dos documentos. Nessa etapa, os documentos são carregados, limpos e divididos em chunks. Cada chunk é transformado em um vetor por meio de um modelo de embeddings. Esses vetores são armazenados em um banco vetorial.

Quando o usuário faz uma pergunta, a pergunta também é transformada em vetor. O sistema compara o vetor da pergunta com os vetores dos chunks armazenados. Os chunks mais parecidos são recuperados e usados como contexto para o modelo gerador.

Bancos vetoriais são ferramentas especializadas em armazenar e buscar vetores. Eles permitem fazer buscas por similaridade de forma eficiente. Exemplos comuns incluem FAISS, Chroma, Pinecone, Weaviate e Milvus.

A avaliação de um sistema RAG envolve diferentes dimensões. Uma delas é a qualidade da recuperação: os chunks recuperados são realmente relevantes? Outra dimensão é a fidelidade da resposta: o modelo respondeu usando o contexto ou inventou informação? Também é importante avaliar se a resposta é completa, clara e útil para o usuário.

Apesar de reduzir o risco de alucinação, RAG não elimina completamente esse problema. Se a recuperação trouxe contexto errado, incompleto ou irrelevante, o modelo ainda pode gerar uma resposta ruim. Por isso, bons sistemas RAG dependem de bons documentos, bons chunks, bons embeddings, boa recuperação e boa avaliação.

```
chunks_material = criar_chunks_por_palavras(
    material_aula_4,
    tamanho_chunk=70,
    sobreposicao=15
)

df_material = pd.DataFrame([
    {
        "chunk_id": f"material_chunk_{i+1}",
        "doc_id": "material_aula_4",
        "titulo": "Material Aula 4",
        "chunk_index": i + 1,
        "texto": chunk
    }
    for i, chunk in enumerate(chunks_material)
])

df_material
```

```
store_material = MiniVectorStore()
store_material.fit(df_material)

store_material.search("RAG elimina completamente alucinação?", top_k=3)
```

Exercício 8 — Perguntas sobre o material maior

Use o `store_material` para responder:

1. Por que a janela de contexto é uma limitação?
2. Como funciona a etapa de indexação?
3. Qual é o papel dos bancos vetoriais?
4. RAG elimina alucinação?
5. Quais dimensões podem ser usadas para avaliar RAG?

Depois, analise se os chunks recuperados foram suficientes.

```
# TODO: teste as perguntas do exercício aqui

pergunta = "?"

store_material.search(pergunta, top_k=3)
```

Parte 12 — Desafio final

Agora é sua vez.

Você vai construir um mini-RAG para responder perguntas sobre um texto.

Você pode usar:

- o texto `material_aula_4`;
- outro texto fornecido pelo professor;
- ou um trecho copiado de slides/PDFs da disciplina.

Sua tarefa é montar o pipeline completo.



```
# DESAFIO FINAL
# Complete o pipeline abaixo.
# O código abaixo é apenas um guideline, pode fazer o seu próprio

texto_desafio = """
Cole aqui um texto maior para testar o seu mini-RAG.
Pode ser um trecho de material didático, documentação, artigo ou conteúdo da aula.
"""

# 1. Criar chunks
chunks_desafio = criar_chunks_por_palavras(
    texto_desafio,
    tamanho_chunk=60,
    sobreposicao=10
)

# 2. Montar DataFrame
df_desafio = pd.DataFrame([
    {
        "chunk_id": f"desafio_chunk_{i+1}",
        "doc_id": "texto_desafio",
        "titulo": "Texto do Desafio",
        "chunk_index": i + 1,
        "texto": chunk
    }
    for i, chunk in enumerate(chunks_desafio)
])

# 3. Indexar no mini vector store
store_desafio = MiniVectorStore()
store_desafio.fit(df_desafio)

# 4. Criar perguntas
perguntas_desafio = [
    "Pergunta 1 aqui",
    "Pergunta 2 aqui",
    "Pergunta 3 aqui",
    "Pergunta 4 aqui",
    "Pergunta 5 aqui"
]

# 5. Rodar recuperação
for pergunta in perguntas_desafio:
    print("=" * 100)
    print("Pergunta:", pergunta)
    print("=" * 100)

    display(store_desafio.search(pergunta, top_k=3))
```

Reflexão final

Responda com suas palavras:

- 1. O que é RAG?
- 2. Por que chunking é importante?
- 3. Qual é o papel dos embeddings?
- 4. O que um banco vetorial faz?
- 5. Por que RAG não elimina completamente alucinação?
- 6. O que você avaliaria antes de colocar um sistema RAG em produção?

Fechamento

Nesta prática, a gente construiu um mini-RAG do zero.

Mesmo sem usar um LLM externo, passamos pelas partes principais:

- base de conhecimento;
- chunking;
- vetorização;
- busca por similaridade;
- recuperação de contexto;
- montagem de prompt;
- resposta baseada em contexto;
- avaliação simples.

Esse é o coração de um sistema RAG.

Ferramentas como LangChain, LlamaIndex, Chroma, FAISS e APIs de LLMs ajudam a escalar esse processo.

Mas a lógica central continua sendo a mesma:

```
1 buscar primeiro, responder depois.
```